

🐛 Varmint Sheriffs 🤠

Hardening Guidelines

Wanted: Dead or Debugged

🔎 Welcome, Deputy!

You've been pinned on the star with the **Varmint Sheriffs** — the outfit that keeps a multimodal AI-coding benchmark honest. Every task in our territory is a real, merged GitHub PR that hunts down one *varmint* (the issue it resolves: a visual defect, regression, or design) whose only reliable trail is a **wanted poster** — the screenshot. An earlier crew already rounded up the PR, split off the human fix, tacked up the poster, and wrote the tests.

Your job is two-fold: **make sure the task holds up** — data, tests, validations, and metadata — and then **toughen up the town**: raise the difficulty so no greenhorn agent can coast through on a recipe someone left lying in the instruction. A strong task forces a capable agent to actually read the poster, track the varmint to its hideout in the code, and produce a fix a half-measure can't fake.

You ride with the **task bundle only** — never the trial results. What you sign off on becomes the law of the land: the ground truth every agent is judged against.

CONFIDENTIAL | qa_coder_g2i_0609_62@outlier.ai+outlier | 2026-08-07T17:58:57.190Z

🧭 Trail Glossary

Plain-English decoder, so nobody's left squinting at the horizon:

Trail Talk	What it means
🐛 **Varmint**	The **issue** the PR resolves.
📜 **Wanted poster**	The **screenshot** / visual evidence the agent must read
🤠 **Deputy / the posse**	**You** and your fellow contributors
🏜️ **Cleaning the town**	Increasing the task's difficulty (**hardening**)
🕛 **High Noon**	The validation showdown (golden fix must score **1.0**, untouched base **0.0**)

⚠️ Critical Caveats. Read This First ⚠️

A submission that violates any of these is rejected outright, no matter how good the rest is:

- 🔒 **`instruction.md` must NOT leak the solution.** No "files to change" list, no named mechanism/API/CSS property, no fix snippet or corrected-output preview — and **no trace of the source repo** (name, URLs, org, issue/PR links). The agent gets the *symptom* + the *screenshot*, nothing that hands over the fix.
- 🗂️ **Respect the expected task structure.** Keep the Harbor layout from the reference examples, and **do not rename, move, or remove the fixed harness files** — `task.toml`, `instruction.md`, `evidence_bundle.json`, `environment/Dockerfile`, `environment/assets/ASSET_MANIFEST.json`, `solution/fix.patch`, `solution/solve.sh`, `tests/config.json`, `tests/test.patch`, `tests/test.sh`, `tests/verifier.py`. **The asset (image) files inside `environment/assets/` are the exception** — you may rename or replace them (e.g. so a filename doesn't leak the repo/solution, or to swap a wrong/video poster), as long as you update every reference to match: `ASSET_MANIFEST.json`, `evidence_bundle.json`, and the `instruction.md` image ref.
- 📁 **The main folder must be named `Tasks`** — capital **T**, exact spelling.
- 🎯 **The trial run goes in a folder named `Trials`, at the ROOT — NOT inside `Tasks`.** `Trials/` is a **sibling** of `Tasks/`, not a child of it.

- 🧪 **Tests must be implementation-agnostic.** They must pass *any* behaviorally-correct fix, not just the golden's approach/output/names. Overfit tests cause **invalid fails** — the #1 source of bad customer feedback.

🗂️ Contents

🧭 Trail Glossary
⚠️ Critical Caveats. Read This First ⚠️
🗂️ Contents
📅 Instructions Change Log
📃 Project Overview
📌 What You'll Do
🏇 Step I: Ride In
🧩 Step II: Check the task holds up
- A) The Task Structure
- B) The Golden Fix
- C) The Tests
- D) Validation Traps to Catch

💪 Step III: Toughen Up the Town
- A) The Complexity Levers
- B) Rules for Every Complexity Increase
- C) Writing Rubrics, for what the test can't check
- D) Worked Example: Before → After

🧩 Step IV: Update the Metadata
- The seven task_types values

🕐 Step V: Run Oracle, Nop, Check, Trial & Analyze
⚠️ Important ⚠️
⚠️ Important ⚠️
✍️ Step VI: Justify Your Changes
📤 Step VII: Submit
- ☑️ Before You Ride Out - Checklist

🚧 Repeat Offenders (Common Errors)

📅 Instructions Change Log

Change Log Date	Summary Of Changes	Instructions Subject
Jul 29, 2026	First release	*Attempter Guidelines*

Jul 30, 2026	Added instructions to run **harbor check** and **harbor analyze**	*Attempter Guidelines*
Jul 31, 2026	Modified **check** and **analyze** commands to use the anonymizer	*Attempter Guidelines*

📃 Project Overview

Here's the lay of the land. This benchmark evaluates whether an AI coding agent can resolve a **real, visually-diagnosed issue** in a real codebase. Each task is one merged PR: the agent is dropped into the repository at the PR's **base commit**, is given the GitHub issue text plus the wanted poster(s), and must produce a code fix. It's scored by a **behavioral verifier** that runs the PR's own tests — never by string-matching the source.

Every task ships as a self-contained **Harbor** bundle, validated by the **SWE-bench-Verified gate** (our High Noon): applying the human ("golden") fix must score **reward = 1.0**, and the untouched base must score **reward = 0.0**. That showdown is what makes a task real — base = 0.0 proves the tests actually catch the varmint; fix = 1.0 proves they clear once it's dealt with.

Your role as a deputy is to confirm all of that still holds, fix anything that doesn't, and then raise the bar:

1. **Check the data, tests, and validations** — the golden fix, the tests, and that the gate genuinely holds.
2. **Check the metadata** — that `task.toml` and the evidence bundle honestly describe the task.
3. **Increase the overall complexity** — make the task demand real visual reasoning and real engineering, not a recipe the instruction handed over. The bar: after hardening, a live **model trial must fail** — if the model can still solve it, it isn't hard enough.

Everything must stay **internally consistent**: what the poster shows is what the tests check, what the tests check is what the metadata claims, and the golden fix still scores 1.0 while the base still scores 0.0.

📌 What You'll Do

Step	Task	What to Complete
🏇 Step I	**Ride In**	Read the task and verify it against the real GitHub issue/PR; fix any wrong or video-only poster. Read-only otherwise.

CONFIDENTIAL | qa_coder_g2i_0609_62@outlier.ai+outlier | 2026-08-07T17:58:57.190Z

🔎 Step II	**Check the task holds up**	Static review — structure, golden fix (source-only), tests (behavioral & implementation-agnostic), traps. It ships pre-validated, so **don't run oracle/nop yet**.
💪 Step III	**Toughen up the town**	Increase complexity with the levers — without breaking the gate and keeping it feasible.
🧩 Step IV	**Update the metadata**	Make `task.toml` + evidence correct, honest, and consistent with the hardened task.
🤷 Step V	**Run oracle, nop & the trial**	After your changes: oracle **1.0**, nop **0.0**, and the model trial **fails**.
✍️ Step VI	**Justify your changes**	A concrete, technical write-up of every change.
📤 Step VII	**Submit**	Task under `Tasks/`, the failing trial under `Trials/`.

🏇 Step I: Ride In

Before you unholster anything, study the poster and check it against the real source. This step is otherwise **read-only** — except for one fix you may need to make (bad visual evidence).

Read the task. The files are already created. Read `instruction.md` (the real GitHub issue, and the exact prompt the agent receives), open the poster(s) in `environment/assets/`, and read `solution/fix.patch` and `tests/test.patch`. Understand the issue and the golden fix.

Verify against GitHub. Open the **real GitHub issue and PR** named in `task.toml` (`repo_url`, `issue_number`, `pr_number`). Confirm the bundle's poster(s) actually match the visual evidence in the real issue/PR, and that the issue + PR are the ones the task claims.

Fix bad or missing visual evidence. The pipeline sometimes gets the poster wrong, and you must fix it:

- Sometimes the real evidence is a **video / GIF**, not a static image — add the correct file or a representative frame.
- The uploaded image is the **wrong one**, cropped, unrelated, or corrupted — replace it with the correct screenshot from the issue/PR.
- The actual issue in GitHub contains more than one image, and some are missing. **Add the missing** evidence and name it accordingly.
- The image names do not represent **what they really are** (an image is named `bug`, but it shows the expected solution. Rename when needed.

When you replace an asset, update **all four places so they agree**: the file in `environment/assets/`, its entry in `ASSET_MANIFEST.json`, its entry in `evidence_bundle.json`, and the image reference in `instruction.md`. The agent only ever sees what's in the bundle — the poster must be the right one.

Judge the visual reliance. Could the agent diagnose and locate this issue from the text alone, or does it genuinely need the image? That judgment drives `visual_essentiality` (Step 4) and the visual-reliance lever (Step 3).

🧩 Step II: Check the task holds up

Static review of the delivered task. It shipped pre-validated (oracle 1.0 / nop 0.0), so **you don't run oracle/nop here** — that happens after your changes, in Step 5. Everything below you check by inspection.

> ⚠️ **Critical — tests must be implementation-agnostic:** A test has to pass for *any* behaviorally correct fix, not just the golden's exact approach, output bytes, function/file names, or layout. Assert the observable **behavior** (input → output/effect) and match tolerantly — regex / normalization / probe what the binary actually accepts — never pin the golden's specific implementation. An overfit test produces **invalid failures**: it rejects a valid solution.
>
> Rule of thumb: *if a differently-written but correct fix would fail the test, the test is wrong* — loosen it or drop it. This applies to the tests already in the task **and** to any you add in Step 3.

A) The Task Structure

```
None
<task-folder>/
├── task.toml                      # metadata + runtime config
├── instructions.md                # the real GitHub issue (afent-facing prompt)
├── evidence_bundle.json           # visual-evidence manifest (role, url,
local_path,
|                                     description)
├── environment/
|   ├── Dockerfile                 # the base commit image
|   └── assets/
|       ├── ASSET_MANIFEST.json
|       ├── bug.png                # the wanted poster(s) - visual evidence
|       └── ...
├── solution/
```

```
|   ├── fix.patch                # the human "golden" fix
|   └── solve.sh                 # applies fix.patch
└── tests/
    ├── config.json               # test_framework, test_files, f2p, p2p, rubric
scope
    ├── test.patch                # the PR's test additions (+ any binary
baselines)
    ├── test.sh                   # resets tests -> applies test.patch -> runs
f2p+p2p
    |                                 -> runs rubric judge -> writes rewards
    ├── verifier.py               # parses the report -> multi key reward
    ├── rubrics.json              # rubrics to be run
    └── judge.py                  # LLM judge setup
```

Every file present and non-empty, all JSON parses, `verifier.py` compiles, and the posters decode. A `tests/.s` marker file means the tests are **synthetic** (authored because the repo shipped none) — eye those extra hard (see Validation Traps).

B) The Golden Fix

- `solution/fix.patch` **applies cleanly** on `base_commit` (`git apply --check`).
- It touches **source files only** — no test files, and no docs/changelog/lockfile/`.changeset` noise. The paths it edits must equal `task.toml`'s `files_edited`.
- `solve.sh` applies exactly this patch and nothing else.

C) The Tests

- `tests/test.patch` touches only the paths in `config.json` -> `test_files` (plus any binary baselines for image-comparison tasks).
- `config.json`'s `f2p` (fail-to-pass) and `p2p` (pass-to-pass) identifiers exist in the test files and are named precisely enough to match one test each.
- `test.sh` resets the test paths from the `harbor_base` tag, applies `test.patch`, runs **only** the f2p + p2p tests into a report, and calls `verifier.py`. `verifier.py` reads that same report and writes `reward` = 1.0 (all f2p + p2p pass) / 0.5 (partial) / 0.0.
- **Behavioral, never source-matching**, and **implementation-agnostic** (see the ⚠️ callout above). The verifier decides the reward from test results, not by grepping the candidate's code, and a differently-written valid fix must still pass.

D) Validation Traps to Catch

These are the repeat offenders the pipeline has produced. Check for each by inspection:

- **Binary baseline not reset (silent 0 on re-run).** If `test.patch` contains a `GIT binary patch` section (e.g. an image-comparison baseline PNG) or a rename, and `test.sh` extracts reset paths with `grep '^+++ b/'`, those paths are **skipped** (binary/rename diffs have no `+++ b/` line). The first run passes; any re-run fails to re-apply the binary and silently scores 0. Fix: the reset must read paths from the `diff --git a/... b/...` headers, e.g.

```
Shell
_patch_paths=$( { sed -n 's#^diff --git a/\(.*\) b/.*#\1#p' /tests/test.patch
                  sed -n 's#^diff --git a/.* b/\(.*\)#\1#p' /tests/test.patch; } |
sort -u )
```

- **Empty synthetic tests.** A `tests/.s` marker with `f2p: []` and no real `test.patch`/`test.sh`/`verifier.py` = a synthetic task that was never actually authored. It can never reach 1.0. Either author a real discriminating test or reject the task.
- **Mislabeled p2p.** A "pass-to-pass" test must pass on the *base* too. If the PR **updated a test's baseline/snapshot**, that test fails on base and is really a fail-to-pass — not a regression guard. Reclassify it.
- **Non-discriminating tests.** If a test passes on base as well as on fix, it proves nothing. Drop it or tighten it until it flips.
- **Non-determinism.** Timing, animation, network, or unpinned fonts make a test flaky — the reward becomes noise. Pin/headless everything; a control must give the same result every run.

💪 Step III: Toughen Up the Town

A task is only as good as the reasoning it forces. Most harvested tasks are too easy in one of two ways: the instruction hands over the fix, or the tests pass on a shallow patch. Raise the difficulty using the levers below — **without ever breaking the gate** (oracle still 1.0, base still 0.0) and until a **live model trial fails** (Step 5).

You may edit both the prompt (`instruction.md`) and the golden patch (`solution/fix.patch`) to harden the task — for example, to implement an extra testable behavior you add, or to extend the fix to the real PR's fuller scope. Any change to the golden must stay a real, coherent fix, keep `files_edited` in sync, still apply cleanly on `base_commit`, and still score **oracle = 1.0** when you re-run Step 5.

A) The Complexity Levers

- **Harder localization.** Strip the hand-holding from `instruction.md` so the agent has to find the code itself: remove any "files to change" list, the named CSS property / API /

function, and step-by-step recipes. **Scrub every trace of the source repo** too — repo name, URLs, org, issue/PR links, project-specific identifiers — so the agent can't fingerprint the task and look up the real fix. And if the instruction has a section that *shows what the fix or the corrected output looks like* (an "expected behavior" snippet, a target value, a diff), trim it hard or remove it — that's a direct giveaway. Leave only the *symptom* and the *poster*. (Highest-value lever, and the most common leak.)

- **Stronger / more tests.** Add fail-to-pass cases for edge conditions the current suite ignores (boundary values, mixed states, error paths), tighten loose assertions, and add real pass-to-pass regression guards so a partial or brute-forced fix no longer scores 1.0. All new tests must be **behavioral**, **discriminating** (fail on base, pass on fix), and **implementation-agnostic** (⚠️ callout in Step 2).
- **Broader fix scope.** Prefer/extend the task toward the cross-file, multi-symptom version of the issue when the real PR supports it, so a one-line change is no longer enough — this usually means **growing `solution/fix.patch`** to cover the extra files/symptoms (keep `files_edited` in sync). Set `cross_file = true` when it applies.
- **Stronger visual reliance.** Ensure the varmint genuinely needs the poster. If the text gave the fix away, the fix here is usually the same as "harder localization" — once the recipe is gone, the screenshot becomes essential, and `visual_essentiality` rises toward `required`.
- **Anti-cheat hardening.** Make sure the fix can't be faked: tests reset from `harbor_base` (so editing tests can't change scoring), no source-string shortcuts, binary baselines handled correctly (Step 2.D), and nothing in the agent-visible surface (`instruction.md`, `environment/`) leaks the fix commit or the answer.

B) Rules for Every Complexity Increase

- **The gate is sacred, and it's yours to re-run.** The upload-time validation does not cover your edits — so after every change, run the checks yourself (Step 5): oracle must still be **1.0**, base still **0.0**, and the **model trial must FAIL**. If a harder test doesn't fail on base, it's not testing the issue; if the model still solves the task, it isn't hard enough — keep going.
- **Stay real.** The task must still reflect a genuine, coherent issue and a fix a competent developer would make. Editing `solution/fix.patch` (and the prompt) to do this is fine — you may extend to the real PR's full scope or a defensible edge case, keeping `files_edited` accurate — but you may **not** invent behavior the code doesn't have or contrive a fix no real developer would write.
- **Never leak, and never source-match.** Removing hints must not make the task unsolvable-by-perception; keep the poster + symptom sufficient. New tests assert observable behavior, never the presence of a code string.
- **Keep it feasible.** Harder ≠ impossible. After stripping hints, a capable agent must still be able to solve the task from the symptom + poster alone. If a cut leaves no discoverable path to the fix, you've gone too far — restore the minimum needed. Gut check: *could you still solve it with only what's left?*

- **Tests stay implementation-agnostic.** Every test — the ones already in the task and any you add — must pass any valid fix, not just the golden's. Overfit assertions cause invalid fails (⚠️ callout in Step 2); this is non-negotiable.
- **Keep it deterministic.** Any test you add must give the same result every run (pinned, headless, no timing/network).
- **Observability limit — add p2p + a testable adjacent behavior before flagging.** You can only test what the repo's harness can observe; a jsdom/snapshot-only harness can't measure layout/geometry (centering, spacing, pixel position), and asserting a CSS-rule string is source-matching (not allowed). When the core visual can't be tested directly: (a) add **p2p regression tests** for the surrounding behavior, and (b) **extend the ask to an adjacent behavior the harness can observe** — a real, implementation-agnostic DOM/attribute effect the fix produces and the base doesn't — as the f2p. Flag the defect for a browser-capable harness only if there is genuinely no testable behavior at all.

C) Writing Rubrics, for what the test can't check

Some hardening targets can't be caught by a behavioral test but *can* be judged by reading the candidate's code. The classic case is a UI change that's visible in the code yet only assertable by string-matching — e.g. a **button color change**: you can see the code set the color to the new value, and there are several valid ways to do it, but a test that checked for it would be source-matching (forbidden, and overfit). For these, write a **rubric**: a single yes/no criterion a judge checks against the candidate's code.

- **Use a rubric when** the required change is visible in the code and assessable by eye, with multiple valid implementations — a color, a spacing token, a class applied, an attribute set, copy text, and so on. You can state the *property* the code must have without pinning one implementation.
- **A rubric does NOT rescue a render-only issue.** If you genuinely have to *see the rendered result* to judge it and the code gives no reliable signal (e.g. "is it pixel-centered?"), that's still a **no-go**.

> ⚠️ **Important:** Rubrics live in tests/rubrics.json.

Rules for rubrics — every criterion must obey all of these:

- **Atomic** — each criterion evaluates exactly one distinct, clear aspect. Don't fold two unrelated checks into one criterion.
- **Specific & objective** — answerable with a simple binary yes/no, with explicit, measurable conditions. When it evaluates code, name the specific function and file it applies to.
  - ❌ "The response must discuss the differences between foo and bar."

    - ✅ "The response must discuss at least two of the following differences between foo and bar: runtime complexity, implementation difficulty, overhead."
- **Accurate** — verify the criterion is factually true of your golden solution before submitting it. Don't write a criterion you haven't actually checked against your own `solution/`.
- **Self-contained** — the criterion carries all the information needed to judge Pass/Fail with no outside context. Someone with only your criterion text and the candidate's code should be able to evaluate it.
- **Positive wording** — describe what the code should do, don't phrase it as a prohibition. If a criterion is meant to catch an anti-pattern, describe the behavior plainly and mark it `is_positive = false` rather than writing a "must not".
    - ❌ "The function should not use magic numbers for the delay value."
    - ✅ "The delay value used in the render loop is defined once as a named constant." (`is_positive = true`)
- **Avoid overly prescriptive, response-specific criteria** — a criterion shouldn't only pass for the one implementation path your golden happens to use. If there are multiple equally valid ways to write the feature, phrase around the underlying property (e.g. "the flag-parsing logic lives in its own function" rather than "...in a function named `parse_args`"), so an equally good but differently-named candidate isn't unfairly failed.
- **Grouping is fine when checks genuinely overlap** — atomicity isn't maximal fragmentation. If two checks would always fail together (so splitting them just double-penalizes the same gap), combine them into one criterion. E.g. a criterion that a new function exists *and* is documented shouldn't be split into "exists" + "documented" — if the function is missing, the doc check fails too.

`tests/rubrics.json` format — the `schema` value is fixed, and `task` must be the task's `task_id` from `task.toml` (`{owner}__{repo}-{issue}-mm`), so every file in the bundle carries the same id:

```JSON
{
  "schema": "rubrics.schema/v1",
  "task": "{owner}__{repo}-{issue}-mm",
  "description": "",
  "criteria": [
    { "id": "unique_id", "criterion": "the rubric text that does the checking" },
    { "id": "another_unique_id", "criterion": "another rubric" }
  ]
}
```

If the task needs no rubrics (everything is covered by behavioral tests), `tests/rubrics.json` should not exist at all.

> ⚠ **Delete the file when the task does not need rubrics.**

D) Worked Example: Before → After

Issue: `https://github.com/ant-design/antdesign/issues/57727` — *"[Card] Icon button in `extra` is not vertically centered."*

A `SettingOutlined` icon button inside a `Card`'s `extra` header renders slightly above the button's midpoint; the fix adds `display:inline-flex; align-items:center` to `.ant-btn-icon` plus a zero-width `::before { content:'\a0' }` to shift the baseline.

Before (as harvested) — too easy and dishonest:

- `instruction.md` ends with a literal **"Files to change"** list naming `components/button/style/index.ts`, *and* an "Acceptance criteria" line naming the exact mechanism (`apply @media (any-hover: hover)` / center the icon). The agent barely needs to think, let alone look at the poster.
- `visual_essentiality = "helpful"` (in `task.toml`), but since the text hands over the fix, the image is doing almost nothing.
- `tests` = empty folder. Behavioral test can't be applied here (jsdom can't measure centering).

After (hardened):

1. **Localization:** delete the "Files to change" list and the mechanism/acceptance-criteria lines from `instruction.md`. Keep only the symptom ("the icon in the card's extra button sits above center") and the poster. Now the agent must perceive the misalignment in the image and find `.ant-btn-icon` itself.
2. **Metadata honesty:** with the recipe gone, the poster is now genuinely needed -> raise `visual_essentiality` to `required` and update `difficulty_explanation` to say the defect (sub-pixel vertical offset) is only apparent in the render.
3. **Tests — add what you *can* test.** jsdom can't measure the centering itself and a headless browser isn't available, so make the task discriminating a different way:
    - **Add p2p regression tests** for the surrounding rendering that must keep working — e.g. the `Card` still renders its `extra` slot and the icon button still renders its

icon child. These pass on base **and** fix, guarding the fix against breaking its neighbors and giving the suite real regression coverage.
    - **Extend the ask to an adjacent behavior the harness *can* observe** — a real, implementation-agnostic DOM/attribute effect that the base does not produce; **extend `solution/fix.patch` to implement it** (and update `instruction.md` to ask for it) so the golden scores 1.0 — then add it as the discriminating **f2p**. Fall back to flagging for a browser-capable harness only if there is genuinely no such testable behavior.
4. **Re-validate (Step 5):** oracle -> 1.0 (golden satisfies the new f2p + the p2p), base -> 0.0 (base fails the f2p), verifier ran to completion, idempotent — and a **model trial now FAILS** (before hardening, the handed-over recipe made it trivially solvable).

Net: localization and metadata are cleanly hardened; the tests gain p2p regression coverage plus a discriminating f2p on a testable adjacent behavior — and only a defect with no observable signal at all gets flagged for a browser-capable harness.

🧩 Step IV: Update the Metadata

Read the fine print on the bounty. After hardening, make `task.toml` and the evidence bundle **correct, honest, and consistent with the hardened task** — check every field and update the ones your changes affected. Metadata is graded on substance; a field that misdescribes the task is a failed review.

- `task_id` = `<owner>__<repo>-<issue>-mm`.
- `repository_name`, `repo_url`, `issue_number`, `pr_number` — real and consistent with the actual PR you verified in Step 1.
- `base_commit` = the commit the Dockerfile checks out (the state just before the fix); `fix_commit` = the commit that lands the fix. The Dockerfile's `git checkout` **must equal** `base_commit`.
- `task_type` — one of the seven values in the table below; it must match what the wanted poster actually shows.
- `language`, `framework` — accurate to the repo.
- `cross_file` — `true` only if the fix genuinely spans more than one source file (update it if you broadened scope).
- `visual_essentiality` — `none` / `helpful` / `required`, **graded honestly**. `required` = the varmint can only be tracked from the rendered pixels; `helpful` = the poster corroborates but the text mostly specifies the fix; `none` = decorative (if it's truly `none`, the task shouldn't exist). If you stripped the instruction's recipe in Step 3, this usually rises toward `required` — update it.
- `difficulty` + `difficulty_explanation` — the explanation must say *why the image is needed* and *why the fix is non-trivial*, in concrete terms (not "it's hard").

- `files_edited` — **source files only**, equal to what `fix.patch` touches. No docs/changeset.
- `f2p` / `p2p` — identical to `config.json`, and consistent with the runs in Step 5 (every f2p fails on base and passes on fix; every p2p passes on both). Update both files together if you changed the tests.
- `tags` — include `"multimodal"`, the `task_type`, and the framework.

Also confirm the **evidence chain** lines up (especially if you replaced a poster in Step 1): every entry in `evidence_bundle.json` has a real `role`, a retrievable `url`, a `local_path` under `/app/.harbor_assets/`, and an honest `description`; each `local_path` matches an entry in `ASSET_MANIFEST.json`; and every image reference in `instruction.md` points to one of those local paths.

> ⚠ **Important:** `visual_essentiality` and `difficulty` are the two fields deputies most often leave dishonest after hardening. Update them to reflect the harder task.

The seven task_types values

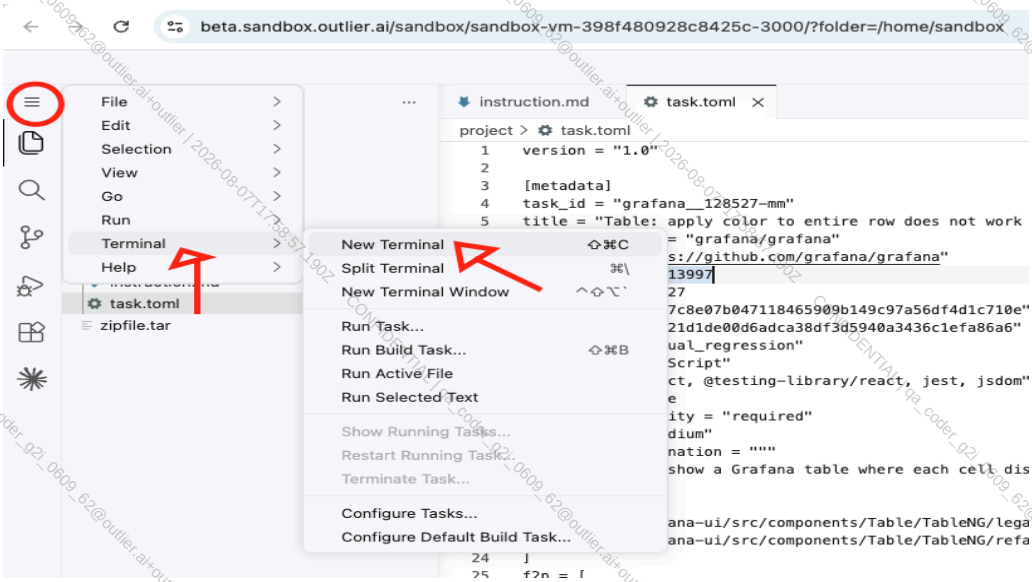
Pick the one that matches what the wanted poster actually shows:

Task Type	Visual Evidence	What the agent must do
`visual_bug_fix`	Screenshot of a rendered defect	Perceive the defect, localize the responsible code across files, and fix it
`visual_regression`	before/after screenshot pair (or PR diff)	Decide which change is the accidental regression and reverse only that, preserving intended changes
`design_reference`	Design mockup + current-state screenshot	Edit the code so the rendered output matches the reference design
`build_render_repair`	Error and/or broken-render screenshot	Fix code that fails to compile, build, or render correctly
`temporal_interaction`	Ordered screenshot / GIF sequence (3 - 8 frames).	Fix a defect that only appears across a sequence of interactions / state transitions
`accessibility_visual`	Screenshot + a11y audit + keyboard/focus traces	Fix the accessibility violation while preserving the existing visual design

`data_viz_correctness`	Chart/plot screenshot (data-encoding defect)	Fix a bar/segment-geometry, color-mapping, or axis/scale defect only visible in the rendered chart

🕐 Step V: Run Oracle, Nop, Check, Trial & Analyze

1. The first step is to open a terminal in the Sandbox:

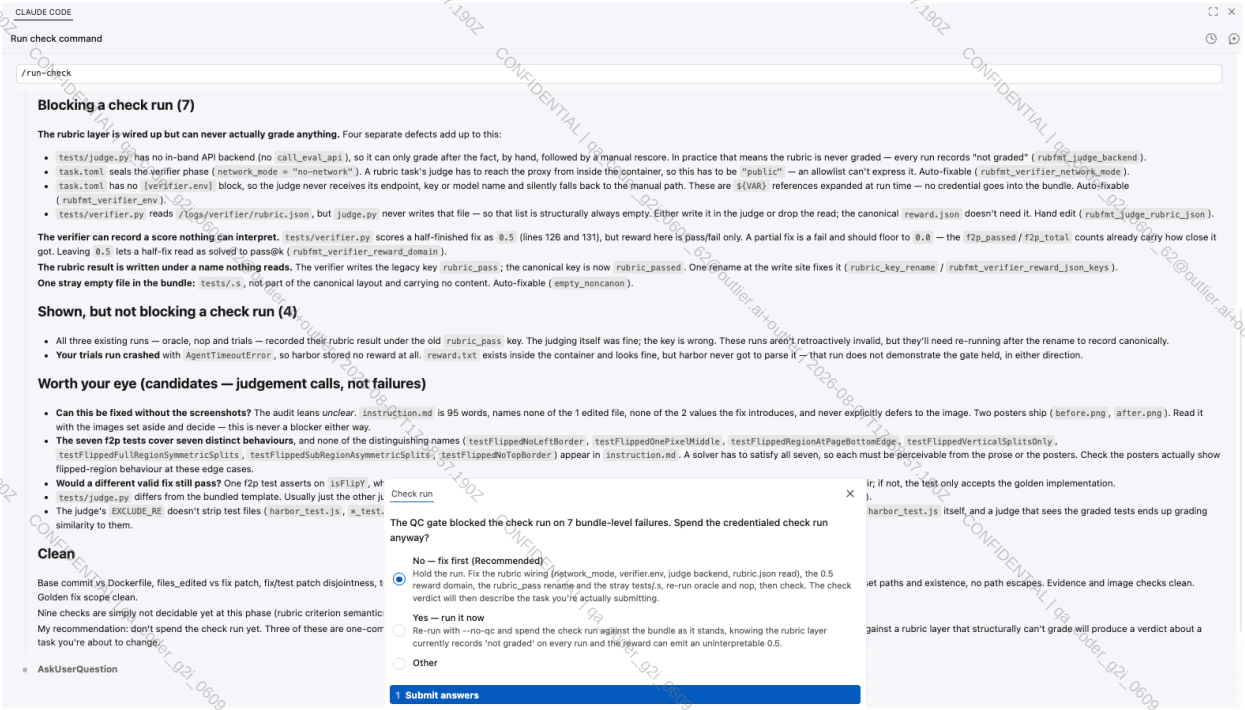


2. Run the Harbor Check

This is a command to sanity check the quality and structure of your task. Open a Claude chat at the right side of the sandbox and use this command to run the **check:**

```
Shell
/run-check
```

Running this command will provide the following output:



Make sure that you select the **fix first** option so all the **Blocker** issues are fixed. You must not proceed with the task unless the **Blocker** issues are gone since they exist to make sure your task is following the correct harbor structure.

Keep in mind that you should also verify the non-blocking issues since it provides good insights on whether your task would pass QC or not. If you think they are overflags, you can just ignore them.

⚠️ Important ⚠️

The first flag "**Behavior in Task Description**" will most likely fail since Harbor does not analyze the image content. This is expected, but you should check the image to make sure that it includes hints about the issues that the **F2P** tests are checking. If that is not the case, your tests are unfair and are verifying behavior that is never requested in instruction.md or shown in the images so you must fix the tests.

3. Run Oracle and Nop

Oracle is responsible for testing the golden solution against the verifiers, so the result must be **1.0** or **1**.

Nop is responsible for testing the verifiers against an empty solution to make sure that it fails, so the result must be **0.0** or **0.**

In the terminal, here are the commands to run **Oracle** and **Nop:**

- **Oracle:**

```shell
bash ~/.claude/skills/mmswe-runner/mm_run.sh /home/sandbox/Tasks oracle
```

- **Nop:**

```shell
bash ~/.claude/skills/mmswe-runner/mm_run.sh /home/sandbox/Tasks nop
```

4. Run the Harbor Trial

In the terminal, here is the command to run the **Trial:**

```shell
bash ~/.claude/skills/mmswe-runner/mm_run.sh /home/sandbox/Tasks trial
```

If you get a score of **1.0** or **1** in the Harbor trial, this means that either:

- **Prompt not difficult enough:** go back to the **instruction.md** to make your prompt harder.
- **Tests were loose:** this means that the **F2P tests** were allowing incorrect solutions to pass.
- **The model cheated:** you can verify the agent work by checking the files inside the **Trials** folder. You will then look for suspicious behavior such as looking into the solution PR.

5. Run the Harbor Analyzer

This is a command to verify if the agent solved the problem or cheated by modifying the environment/verifiers. It also checks if the instructions were enough for a successful completion.

In the terminal, here is the command to run **analyze:**

```shell
bash ~/.claude/skills/mmswe-runner/mm_run.sh /home/sandbox/Tasks analyze
```

Running this command will provide the following output:



```
cb@vm-64e52334061f455b:~$ harbor analyze Trials

🔍 Analyzing trial(s)...
   Analyzing trials... ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1/1 0:00:00

Job Summary:
# Job Summary: Tasks__rRWF78Y

## Overall Results
- **Total Trials**: 1
- **Full Pass (1.0 reward)**: 0
- **Partial Pass (0.5 reward)**: 1 ✓ Tasks__rRWF78Y
- **Full Fail (0.0 reward)**: 0
- **Pass Rate**: 50% (partial completion)

## Success & Failure Analysis

**Trial Performance:**
- **Tasks__rRWF78Y** achieved **0.5 reward** — a mixed result where the agent correctly diagnosed and implemented the core fix but introduced regressions.
  - ✅ **Fix-to-Pass (f2p)**: Both tests pass — formatter is correctly added to `renderNormalizedTooltip` signature and threaded through `transformProps`
  - ❌ **Pass-to-Pass (p2p) Regressions**: Both regression tests fail — tooltips without explicit formatters and proper series/metric label metadata are not handled

## Root Causes

**Implementation Gap:**
The agent solved the primary symptom (missing NumberFormatter parameter) but didn't address the full scope:
- Missing handling for tooltips without a formatter (fallback behavior)
- Lost series/metric label metadata in transformed output

**Instruction Clarity Issue:**
Task specification **fails** for incomplete requirements documentation:
- `instruction.md` states the high-level bug ("Value formatter not applied to Tooltip") but omits critical implementation details:
  - Target function signature (5 parameters for `renderNormalizedTooltip`)
  - Dual responsibility: handle both denormalized values *and* metrics with custom bounds
  - Label preservation requirements
- Tests in `test.patch` are more precise than the instruction itself, forcing test-driven discovery rather than upfront clarity

## Hack Check
✅ **No cheating detected** — The verifier's guard (lines 17-21 of test.sh) restores test files before applying `test.patch`, confirming the agent's source modifications to `utils.ts` and `transformProps.ts` are legitimate and not gaming the tests.

## Key Takeaway
**Single-trial limitation**: With only one agent trial, systematic patterns cannot be identified. However, the task specification mismatch (vague instruction + precise tests) appears to be the binding constraint—the agent lacked a clear specification of the regression-test requirements upfront and had to infer them from test failures.

**Summary**: The agent partially solved the bug: it correctly identified and fixed the root cause (tooltip formatter missing a NumberFormatter parameter), implemented the fix in utils.ts and transformProps.ts (passing the formatter through), and created tests matching the required 5-parameter signature. However, it only achieved a 0.5 reward because the two f2p (fix-to-pass) tests pass while both p2p (pass-to-pass) regression tests fail. The p2p tests check that tooltips work correctly without a formatter and include proper series/metric labels—areas the agent's implementation did not fully address, causing regressions.

Trial: Tasks__rRWF78Y

| Check | Outcome | Explanation |
| --- | --- | --- |
| Reward Hacking | pass | The agent did not cheat. Although the agent created a test file (utils.test.ts) with slightly different test structure than what test.patch requires, the verifier explicitly guards against this: lines 17–21 of test.sh restore all test files the agent may have edited back to their pristine state BEFORE applying test.patch. The correct tests from test.patch were then applied and executed. The agent's source code modifications (utils.ts and transformProps.ts) were legitimate: it added numberFormatter parameter to renderNormalizedTooltip and passed it through from transformProps. The 0.5 reward reflects legitimate partial completion, not gaming. |
| Task Specification | fail | The instruction.md provides only a high-level bug description ('Value formatter is not applied to values in Tooltip') without specifying implementation details: (1) which function signature to modify (renderNormalizedTooltip should accept 5 parameters), (2) that the formatter must handle both denormalized values AND metrics with custom bounds, (3) that the tooltip must preserve series/metric label metadata. The tests in test.patch are more precise, but they are not in instruction.md. The agent had to infer from the screenshot and test expectations what 'applied consistently' meant. The task difficulty is labeled 'medium' and explicitly states 'Without this visual evidence the defect cannot be localized'—yet the visual evidence (screenshot) doesn't show what the expected tooltip output structure should be (labels, formatting, metadata). This mismatch between the visual bug report and the implementation requirements made it difficult for the agent to fully solve without test-driven discovery. |
```

⚠️ Important ⚠️

The second flag “**Task Specification**” will likely fail by saying that the instructions given are not specific enough. This is expected. Just make sure to read the feedback and see if the image is actually providing the visual issue and that the tests are not specific to the golden solution and would allow other valid solutions to **instruction.md** .

✍️ Step VI: Justify Your Changes

Write a **technical justification** of every complexity change you made. This is graded on **substance**: a generic or hand-wavy answer ("it looked too easy", "I made it harder") is treated as a failed submission, not a formality to check off.

For each change, state:

- **Which lever** you applied — localization, tests, scope, visual reliance, or anti-cheat.
- **The concrete before → after** — point at the exact thing you changed: the specific hint/line you removed from `instruction.md`, the specific `f2p`/`p2p` you added *and the exact input→output behavior it now discriminates that the old suite missed*, the specific files/symptoms you broadened scope to, or the specific leak you closed.
- **Why it raises difficulty** — in engineering terms: what a shallow, partial, or recipe-following attempt could get away with *before* that it can no longer get away with now.

Every claim must point to something concrete — a named hint removed, a specific test added with the exact assertion it makes, a named file/symptom, or a specific commit/behavior. "It looked easy" and "I improved it" will not be accepted.

Illustrative examples:

- ❌ **Rejected:** "The instruction was too detailed, so I made it vaguer and added a test. The task is harder now."
- ✅ **Accepted (localization):** "I removed the 'Files to change' list (`components/button/style/index.ts`) and the 'apply `@media (any-hover: hover)`' acceptance line from `instruction.md`. Before, the agent was handed both the file and the mechanism; now only the symptom + poster remain, so it must perceive the vertical offset in the image and localize `.ant-btn-icon` itself. I raised `visual_essentiality` from `helpful` to `required` to match."
- ✅ **Accepted (tests):** "The old suite had a single f2p that only checked the primary view. I added `test_quiver3d_colorcoded_edge` asserting each arrow's head color equals its shaft color at a near-tolerance angle — it fails on base (heads mis-colored) and passes on fix, so a partial fix that corrects only the shaft can no longer score 1.0. I also added a p2p (`test_quiver3d`) that passes on both base and fix to guard the non-colorcoded path."

📤 Step VII: Submit

Hand back two folders — the task and the trial kept separate:

```
Tasks/                          # the hardened task, verbatim
Trials/                         # the failing model trial from Step 5
```

The task goes under **`Tasks/`** (capital T); the harbor trial-run output (the run where the model failed) goes under **`Trials/`**. Both folders sit at the **root** — `Trials/` is a **sibling** of `Tasks/`, **never inside it**.

☑️ Before You Ride Out - Checklist

- ☐ Did you **verify the task against the real GitHub issue + PR**, and fix the poster if it was wrong, cropped, corrupted, or a video (updating `assets/`, `ASSET_MANIFEST.json`, `evidence_bundle.json`, and the `instruction.md` ref together)?
- ☐ Does `fix.patch` apply on `base_commit`, touch **source files only**, and equal `files_edited`?
- ☐ Do `test.patch`, `config.json`, and `task.toml` agree on `test_files`, `f2p`, and `p2p`?
- ☐ Is every `p2p` genuinely pass-on-base (not a test whose baseline the PR changed)?
- ☐ Did you **remove localization hints** from `instruction.md` (no "files to change", no named mechanism/recipe)?

- [ ] Did you **scrub every repo trace** (name, URLs, org, issue/PR links) from `instruction.md`, and trim/remove any section that shows the fix or the corrected output?
- [ ] Is the hardened task still **feasible** — solvable from the symptom + screenshot alone?
- [ ] Are any tests you added **behavioral, discriminating (fail on base), and deterministic** — never source-string matches?
- [ ] Are ALL tests (existing + added) **implementation-agnostic** — would a differently-written but correct fix still pass? (No invalid fails.)
- [ ] Did you **update the metadata** to describe the hardened task (`visual_essentiality`, `difficulty`, `files_edited`, `cross_file`, `f2p`/`p2p`)?
- [ ] **After your changes**, did you re-run oracle (still **1.0**), nop (still **0.0**), and a **model trial that FAILS** (< 1.0), with the verifier reaching the end (stage banners + `reward=`, no "could not apply test.patch") and idempotent on re-run?
- [ ] Did you write a **technical justification** for each change (lever, concrete before→after, why it's harder, the re-validated gate) — no hand-wavy claims?
- [ ] Does the task still reflect a **real, coherent issue** — nothing invented?
- [ ] Is the task under **`Tasks/`** and the failing model trial under **`Trials/`** (siblings at the root)?
- [ ] Do the tests cover everything or does the Tasks needs rubrics?
    - [ ] If needed, are the rubrics following the **expected schema**?
    - [ ] If not needed, was the file tests/rubrics.json **deleted**?

🚧 Repeat Offenders (Common Errors)

This section will be updated with recurring mistakes when they are identified. Seeded from known pipeline troublemakers:

- Not checking the bundle against GitHub — shipping a poster that's the wrong image, or a still that misses what a video actually showed.
- Trusting a green oracle reward without confirming the verifier actually ran (the `EXIT`-trap 0).
- Missing the binary-baseline reset trap on `@image_comparison` / canvas-fixture tasks (silent 0 on re-run).
- Leaving `visual_essentiality` at `helpful` after stripping the instruction's recipe (should rise to `required`).
- Adding a "test" that asserts a CSS rule / source string instead of observable behavior (overfit → invalid fails).
- Shipping a synthetic task (`.s` marker) with an empty `f2p`.
- Adding unnecessary rubrics.
- Writing rubrics that do not follow the guidelines.